



Università degli Studi di Trento

Corso di Tecniche di Programmazione Avanzata ed AI

Anno Accademico 2025/2026

Classificazione di Immagini: Cats vs Dogs

*Confronto tra Architetture CNN Custom e Transfer Learning
con ResNet18*

Studente:

Andrea Alfredo Zambelli

Matricola: 245311

Docente:

Prof. Paolo Rech

Indice

1	Introduzione e Background Teorico	2
1.1	Obiettivi del Progetto	2
1.2	Dal Machine Learning al Deep Learning	2
1.3	Le Reti Neurali Convoluzionali (CNN)	2
1.4	Problematiche comuni: Underfitting e Overfitting	2
2	Analisi e Preparazione del Dataset	4
2.1	Origine dei Dati: Il Dataset PetImages	4
2.2	Integrità dei Dati e Pre-processing	4
2.3	Data Augmentation	4
2.4	Normalizzazione dei Tensori	5
2.5	Suddivisione del Dataset: Training e Test Set	5
3	Architettura dei Modelli e Scelte Progettuali	6
3.1	Modello 1: Custom CNN	6
3.1.1	Implementazione del Modello Custom	6
3.1.2	Analisi dei Layer e delle Scelte Progettuali	7
3.2	Modello 2: Transfer Learning con ResNet18	8
3.2.1	Feature Extraction e Freezing	8
3.3	Sintesi delle Differenze	8
4	Processo di Addestramento e Ottimizzazione	9
4.1	Funzione di Costo: BCE Loss	9
4.2	Ottimizzatore: Algoritmo Adam	9
4.3	Protocollo di Training e Validazione	10
5	Analisi dei Risultati e Confronto delle Performance	11
5.1	Andamento del Training e Accuratezza Globale	11
5.2	Analisi Critica delle Matrici di Confusione	12
6	Conclusioni	14

1 Introduzione e Background Teorico

1.1 Obiettivi del Progetto

Il presente elaborato descrive lo sviluppo e il confronto di modelli di Deep Learning applicati alla classificazione di immagini. Nello specifico, il compito richiesto consiste nel distinguere automaticamente tra due classi di animali domestici: cani e gatti.

L'obiettivo primario non è soltanto il raggiungimento di un'elevata accuratezza, ma l'analisi critica del comportamento di diverse architetture di reti neurali convoluzionali (CNN), valutando come il pre-processamento dei dati e l'utilizzo di tecniche di Transfer Learning possano influenzare le performance finali.

1.2 Dal Machine Learning al Deep Learning

Come introdotto nelle prime lezioni del corso, il *Machine Learning* classico si basa sullo sviluppo di algoritmi statistici capaci di apprendere dai dati. Tuttavia, per compiti complessi come la visione artificiale, il ML classico mostra dei limiti dovuti alla necessità di estrarre manualmente le caratteristiche (feature engineering).

Il *Deep Learning*, un sottoinsieme del Machine Learning, supera questo ostacolo utilizzando reti neurali stratificate. Questi modelli sono in grado di estrarre automaticamente feature di alto livello direttamente dai dati grezzi (i pixel delle immagini), rendendoli lo standard de facto per la classificazione di immagini.

1.3 Le Reti Neurali Convoluzionali (CNN)

Le CNN rappresentano l'architettura fondamentale per questo progetto. A differenza delle reti neurali tradizionali, le CNN preservano la struttura spaziale delle immagini trattandole come griglie 2D con tre canali colore (RGB).

I componenti chiave implementati nel codice e che verranno analizzati nei capitoli successivi sono:

- **Layer Convoluzionali:** Utilizzano filtri (kernel) per identificare pattern locali come bordi, texture e forme.
- **Funzioni di Attivazione (ReLU):** Introducono non-linearità nel modello, permettendo alla rete di apprendere relazioni complesse.
- **Subsampling (Max Pooling):** Riduce la dimensionalità spaziale delle mappe di feature, rendendo il modello più robusto a piccole traslazioni dell'input e riducendo il carico computazionale.
- **Layer Fully Connected:** Situati alla fine della rete, aggregano le feature estratte per produrre la classificazione finale tramite una funzione *Sigmoid* per l'output binario.

1.4 Problematiche comuni: Underfitting e Overfitting

Durante lo sviluppo dei modelli di Deep Learning, l'obiettivo centrale è la massimizzazione della capacità di generalizzazione statistica. Particolare attenzione è stata quindi posta

al bilanciamento della capacità della rete, monitorando costantemente l'equilibrio tra due fenomeni speculari:

1. **Underfitting:** Si verifica quando il modello presenta una capacità espressiva insufficiente (architettura troppo leggera) o un tasso di apprendimento che non gli consente di convergere. In questo scenario, la rete non riesce a catturare le relazioni geometriche fondamentali dei dati, mostrando performance insoddisfacenti (alta loss e bassa accuratezza) sia nel set di training che in quello di test.
2. **Overfitting:** Si verifica quando il modello possiede un'elevata complessità parametrica e finisce per “memorizzare” i dettagli specifici del dataset di addestramento. Ciò si traduce graficamente in una curva di training loss che decresce verso lo zero, a cui corrisponde però un netto peggioramento delle performance e una forte divergenza dell'accuratezza sulle immagini mai viste nel test set.

Per contrastare l'insorgenza dell'overfitting e forzare i modelli a estrarre feature visive invarianti, in questo progetto sono state adottate strategie di **Data Augmentation** e tecniche di regolarizzazione strutturale, che verranno approfondite nei capitoli successivi.

2 Analisi e Preparazione del Dataset

2.1 Origine dei Dati: Il Dataset PetImages

Per questo progetto è stato utilizzato il dataset *Dogs vs. Cats* distribuito da Microsoft Research e comunemente reperibile su Kaggle. Il dataset originale comprende 25.000 immagini di cani e gatti, suddivise equamente tra le due classi.

La scelta di questo dataset presenta sfide reali tipiche della computer vision:

- Immagini di dimensioni eterogenee.
- Condizioni di illuminazione variabili.
- Soggetti ripresi da diverse angolazioni e in contesti differenti (interno/esterno).

2.2 Integrità dei Dati e Pre-processing

Una fase critica, spesso sottovalutata ma fondamentale per la stabilità del training, è la pulizia del dataset. I dataset reali possono contenere file corrotti o non leggibili che causerebbero il crash dei processi di caricamento di PyTorch.

È stata implementata una funzione di *Data Cleaning* dedicata che scansiona le directory, apre ogni immagine con la libreria PIL e ne verifica l'integrità. I file che non superano il test vengono automaticamente rimossi prima dell'inizio dell'addestramento.

2.3 Data Augmentation

Per migliorare la capacità di generalizzazione del modello e contrastare l'overfitting, ho applicato tecniche di *Data Augmentation* esclusivamente al set di addestramento. L'idea teorica è quella di "ingannare" la rete facendole credere di avere un dataset molto più vasto e variegato.

Le trasformazioni applicate tramite il modulo `torchvision.transforms` includono:

1. **RandomHorizontalFlip**: Specchiatura orizzontale casuale con probabilità del 50%.
2. **RandomRotation**: Rotazione casuale dell'immagine entro un raggio di 10 gradi.
3. **Resize**: Ridimensionamento uniforme a 128×128 pixel per garantire tensori di forma costante in ingresso alla rete.

Sperimentalmente, il confronto tra il modello addestrato con e senza data augmentation ha mostrato variazioni minime nell'accuratezza finale sul test set. Tale comportamento è giustificato principalmente dall'elevata dimensionalità iniziale del dataset (circa 20.000 immagini nel training set al netto della pulizia), la cui ricchezza informativa riduce la dipendenza immediata da campioni sintetici. Inoltre, l'addestramento limitato a sole 5 epoche ha permesso una rapida convergenza prima che potesse manifestarsi un overfitting marcato.

2.4 Normalizzazione dei Tensori

Un aspetto determinante per il successo del modello, specialmente per l'architettura ResNet18, è la normalizzazione dei canali RGB. I pixel sono stati inizialmente scalati nell'intervallo $[0, 1]$ e successivamente normalizzati utilizzando i parametri di ImageNet:

$$X_{norm} = \frac{X - \mu}{\sigma} \quad (1)$$

Dove $\mu = [0.485, 0.456, 0.406]$ e $\sigma = [0.229, 0.224, 0.225]$. Questa operazione garantisce che la distribuzione dei dati di input sia coerente con quella utilizzata durante il pre-addestramento della rete neurale.

Di seguito viene riportato lo snippet di codice Python utilizzato per definire le pipeline di trasformazione. Si noti la distinzione tra il set di addestramento (dove viene applicata l'augmentation) e il set di test.

Listing 1: Configurazione di Data Augmentation e Normalizzazione

```
1 # Trasformazioni per il Training Set (con Augmentation)
2 train_transform = transforms.Compose([
3     transforms.Resize((128, 128)),
4     transforms.RandomHorizontalFlip(p=0.5),      # Riflessione
5     transforms.RandomRotation(degrees=10),       # Rotazione
6     transforms.ToTensor(),                       # Conversione in
7     transforms.Normalize(                        # Normalizzazione
8         ImageNet
9         mean=[0.485, 0.456, 0.406],
10        std=[0.229, 0.224, 0.225]
11    )
12 ])
13 # Trasformazioni per il Test Set (solo pre-processamento base)
14 test_transform = transforms.Compose([
15     transforms.Resize((128, 128)),
16     transforms.ToTensor(),
17     transforms.Normalize(
18         mean=[0.485, 0.456, 0.406],
19         std=[0.229, 0.224, 0.225]
20     )
21 ])
```

L'inclusione di `RandomHorizontalFlip` e `RandomRotation` permette di esporre la rete a varianti diverse della stessa immagine ad ogni epoca, forzando l'estrazione di feature invarianti rispetto alla posizione e all'orientamento del soggetto.

2.5 Suddivisione del Dataset: Training e Test Set

Il dataset è stato suddiviso in due sottoinsiemi indipendenti utilizzando la funzione `random_split` di PyTorch:

- **Training Set (80%):** Utilizzato per l'aggiornamento dei pesi tramite backpropagation.
- **Test Set (20%):** Utilizzato esclusivamente per la valutazione finale delle performance su dati mai visti durante il training.

Questa separazione è essenziale per verificare che il modello non stia semplicemente "memorizzando" le immagini di addestramento, ma stia effettivamente imparando a riconoscere le caratteristiche morfologiche di cani e gatti.

3 Architettura dei Modelli e Scelte Progettuali

In questo capitolo vengono analizzate le due diverse filosofie di design adottate per risolvere il compito di classificazione: la creazione di una rete neurale *da zero* e l'adattamento di un modello tramite Transfer Learning.

3.1 Modello 1: Custom CNN

La CustomCNN è stata progettata seguendo i principi delle architetture classiche come la LeNet-5. L'obiettivo era creare una rete leggera ma profonda a sufficienza per estrarre feature discriminanti (bordi, forme, texture) tipiche dei musci di cani e gatti.

3.1.1 Implementazione del Modello Custom

Il modello è suddiviso in due blocchi logici principali, seguendo lo standard delle reti neurali convoluzionali moderne analizzate a lezione: un estrattore di feature (`conv_layers`), deputato alla scomposizione spaziale dell'immagine, e un classificatore fully-connected finalizzato alla discriminazione statistica (`fc_layers`).

Listing 2: Definizione della classe CustomCNN per immagini 128×128

```

1 class CustomCNN(nn.Module):
2     def __init__(self):
3         super(CustomCNN, self).__init__()
4         # Estrattore di feature: Convoluzioni e
          Sottocampionamento (Subsampling)
5         self.conv_layers = nn.Sequential(
6             nn.Conv2d(3, 16, kernel_size=3, padding=1), nn.ReLU(),
              nn.MaxPool2d(2, 2),
7             nn.Conv2d(16, 32, kernel_size=3, padding=1), nn.ReLU(),
              nn.MaxPool2d(2, 2),
8             nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.ReLU(),
              nn.MaxPool2d(2, 2)
9         )
10        # Classificatore finale: Layer Fully Connected
11        self.fc_layers = nn.Sequential(
12            nn.Flatten(),
13            nn.Linear(64 * 16 * 16, 128), nn.ReLU(),
14            nn.Linear(128, 1),

```

```

15         nn.Sigmoid() # Output probabilistico binario
16     )
17
18     def forward(self, x):
19         x = self.conv_layers(x)
20         return self.fc_layers(x)

```

3.1.2 Analisi dei Layer e delle Scelte Progettuali

- **Convolutional Layers e Canali Gerarchici:** I tre strati di convoluzione bidimensionale utilizzano filtri (kernel) di dimensioni 3×3 . Questa dimensione rappresenta uno standard ingegneristico ottimale per catturare relazioni spaziali locali minimizzando il numero di parametri da addestrare rispetto a kernel più grandi (es. 5×5).

Il numero di filtri estrattori raddoppia progressivamente (da 16 a 32, fino a 64). Questa stratificazione asseconda la gerarchia visiva tipica del Deep Learning: i primi strati estraggono feature a bassa frequenza e primitive geometriche (bordi, angoli, variazioni cromatiche locali), mentre gli strati più profondi combinano tali primitive per mappare configurazioni semantiche complesse (struttura del muso, texture del pelo). L'uso del parametro `padding=1` assicura che il perimetro dell'immagine non perda informazioni durante l'operazione di convoluzione, mantenendo invariata la risoluzione spaziale prima del sottocampionamento.

- **Funzione di Attivazione ReLU (Rectified Linear Unit):** Interposta dopo ogni operazione convoluzionale e lineare, la funzione $f(x) = \max(0, x)$ introduce la non-linearità fondamentale per consentire alla rete l'approssimazione di confini di decisione complessi.
- **Classificazione Lineare e Impatto dell'Image Size:** La dimensionalità in ingresso al primo strato lineare dipende direttamente dalla risoluzione geometrica dell'input originario.

Nel setup definitivo impostato a 128×128 pixel, l'output subisce tre dimezzamenti successivi ($128 \rightarrow 64 \rightarrow 32 \rightarrow 16$), producendo un tensore finale pari a $64 \times 16 \times 16 = 16.384$ nodi d'ingresso.

Questa quantificazione geometrica rappresenta il perfetto equilibrio dinamico individuato sperimentalmente durante i test. Infatti, le configurazioni alternative testate hanno evidenziato che:

- Un campionamento superiore a 160×160 inietta 25.600 parametri d'ingresso, fornendo una capacità matematica eccessiva.
- Un sotto-campionamento critico a 96×96 contrae il layer a soli 9.216 nodi, degradando irreversibilmente le feature locali fondamentali e arrestando la convergenza del modello in uno stato latente di underfitting.
- **Output Layer e Funzione Sigmoidale:** Trattandosi di un problema di classificazione binaria pura (Gatto vs Cane), l'architettura converge in un unico neurone

di output lineare. L'applicazione finale della funzione Sigmoidale schiaccia il valore scalare risultante all'interno dell'intervallo continuo $[0, 1]$.

3.2 Modello 2: Transfer Learning con ResNet18

Il secondo approccio sfrutta la **ResNet18**, una rete pre-addestrata su ImageNet. il Transfer Learning è estremamente efficace quando il dataset a disposizione non è vastissimo.

3.2.1 Feature Extraction e Freezing

Per adattare la ResNet al problema, è stata adottata una strategia di *Feature Extraction*. Sono stato "congelati" i layer convoluzionali affinché mantenessero la conoscenza pregressa ed è stata sostituita solo la "testa" (head) della rete.

Listing 3: Adattamento della ResNet18 pre-addestrata

```
1 def get_transfer_model():
2     # Caricamento del modello con pesi pre-addestrati
3     model = models.resnet18(pretrained=True)
4
5     # Freezing: i pesi esistenti non vengono aggiornati
6     for param in model.parameters():
7         param.requires_grad = False
8
9     # Sostituzione dell'ultimo layer FC (Fully Connected)
10    num_ftrs = model.fc.in_features
11    model.fc = nn.Sequential(
12        nn.Linear(num_ftrs, 1),
13        nn.Sigmoid()
14    )
15    return model
```

Dal punto di vista computazionale, l'istruzione `param.requires_grad = False` esclude i parametri nativi della ResNet dal grafo di computazione di PyTorch. Di conseguenza, durante la fase di *backpropagation*, l'ottimizzatore non calcola i gradienti per questi strati, riducendo drasticamente l'onere computazionale e i tempi di addestramento su macchina.

3.3 Sintesi delle Differenze

La differenza fondamentale risiede nel fatto che la **CustomCNN** deve imparare a estrarre feature primitive (come bordi, texture e forme) partendo da un'inizializzazione casuale dei pesi, richiedendo una calibrazione estremamente sensibile degli iperparametri spaziali.

Al contrario, la **ResNet18**, grazie al pre-addestramento sul dataset standard *ImageNet*, possiede già un solido catalogo di filtri visivi complessi e universali. Il Transfer Learning si limita quindi a mappare queste astrazioni preesistenti verso l'ultimo strato lineare, riconfigurando la classificazione per le specificità geometriche di cani e gatti con un rischio di overfitting strutturale pressoché nullo.

4 Processo di Addestramento e Ottimizzazione

L'addestramento di una rete neurale è un problema di ottimizzazione in cui l'obiettivo è trovare l'insieme di pesi e bias che minimizzano una specifica funzione di costo. In questo capitolo vengono descritte le scelte iper-parametriche e il protocollo di training adottato.

4.1 Funzione di Costo: BCE Loss

Dato che il compito è una classificazione binaria (Cane vs Gatto) e l'ultimo layer del modello utilizza una funzione di attivazione *Sigmoid*, la funzione di costo più appropriata è la **Binary Cross Entropy (BCE) Loss**.

Matematicamente, per ogni campione i , la loss è definita come:

$$\mathcal{L} = -[y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)] \quad (2)$$

Dove y_i è l'etichetta reale (0 o 1) e $\hat{y}_i$ è la probabilità predetta dal modello. Questa funzione penalizza pesantemente le predizioni sicure ma errate, guidando il gradiente verso la direzione corretta.

4.2 Ottimizzatore: Algoritmo Adam

Per l'aggiornamento dei pesi è stato scelto l'ottimizzatore **Adam** (Adaptive Moment Estimation) con un learning rate iniziale di 0.001.

A differenza dello Stochastic Gradient Descent (SGD) classico, Adam calcola tassi di apprendimento adattivi per ogni parametro, mantenendo una media mobile sia del gradiente (primo momento) che del suo quadrato (secondo momento). Questa scelta è motivata dalla necessità di una convergenza più rapida in presenza di dataset complessi e architetture profonde come la ResNet18.

La calibrazione dell'iperparametro del *Learning Rate* a 0.001 rappresenta il valore ottimale individuato sperimentalmente per questo specifico setup. Durante la fase di tuning, è stata valutata una configurazione alternativa con un tasso ridotto a 0.0001, la quale ha tuttavia evidenziato forti limiti di convergenza entro il vincolo computazionale delle 5 epoche stabilite:

- Nella **CustomCNN**, un passo pari a 0.0001 ha causato un fenomeno di *underfitting temporaneo*, rallentando la discesa del gradiente lungo la loss function e bloccando l'accuratezza a un valore molto più basso di quello raggiunto con il tasso a 0.001.
- Nella **ResNet18**, pur stabilizzando le micro-oscillazioni sul fondo della valle della loss, un valore così ridotto ha limitato la reattività dell'ultimo strato lineare nell'adattarsi rapidamente alle feature del nuovo dataset.

Di conseguenza, il valore di 0.001 è stato preferito poiché garantisce la massima efficienza nel compromesso accuratezza-tempo, permettendo alla rete da zero di convergere stabilmente e alla testa della ResNet di superare agevolmente il muro del 93% di accuratezza sul Test Set.

4.3 Protocollo di Training e Validazione

L'addestramento è stato eseguito per un totale di 5 epoche stabilite con una dimensione del batch (*batch size*) fissata a 64 campioni per iterazione. Per ogni epoca, il flusso computazionale esegue un duplice ciclo sequenziale:

1. **Training Loop:** Il modello viene configurato in modalità addestramento tramite il flag `model.train()`. Per ogni batch estratto dal `train_loader`, viene eseguito il passaggio in avanti (*forward pass*), calcolato lo scostamento tramite la loss function, azzerati i gradienti accumulati dall'iterazione precedente (`optimizer.zero_grad()`), eseguita la derivazione catenaria (*backpropagation* tramite `loss.backward()`) e, infine, aggiornati i pesi sinaptici mediante lo `optimizer.step()`.
2. **Validation/Test Loop:** Il modello viene commutato in modalità di inferenza statica tramite `model.eval()`. Sfruttando il contesto offerto dal costrutto `torch.no_grad()`, il calcolo del grafo dei gradienti viene completamente inibito. Questa operazione azzerava l'allocazione di memoria supplementare sulla GPU e accelera il tempo computazionale, consentendo di misurare l'accuratezza sul Test Set al termine di ogni epoca per monitorare in tempo reale la capacità di generalizzazione del modello.

Listing 4: Struttura del loop di ottimizzazione PyTorch

```
1 # Configurazione dell'algoritmo di ottimizzazione e della loss
  function
2 optimizer = optim.Adam(model.parameters(), lr=0.001)
3 loss_function = nn.BCELoss()
4
5 for epoch in range(epochs):
6     # Fase di addestramento
7     model.train()
8     for images, labels in train_loader:
9         # Forward pass e rimozione della dimensione superflua (
          squeezing)
10        outputs = model(images).squeeze()
11        loss = loss_function(outputs, labels.float())
12
13        # Fase di ottimizzazione (Backpropagation)
14        optimizer.zero_grad()
15        loss.backward()
16        optimizer.step()
```

5 Analisi dei Risultati e Confronto delle Performance

In questo capitolo vengono presentati e discussi criticamente i risultati sperimentali ottenuti dai due modelli descritti in precedenza. L'analisi si concentra sull'andamento della loss durante le epoche di addestramento, sull'accuratezza globale finale (*Test Accuracy*) e sullo studio dettagliato degli errori di classificazione mediante le matrici di confusione.

5.1 Andamento del Training e Accuratezza Globale

Entrambi i modelli sono stati addestrati per 5 epoche su architettura CPU. L'andamento della funzione di costo e della capacità di generalizzazione sul set di test mostra dinamiche profondamente diverse tra la rete sviluppata da zero e quella basata su Transfer Learning, evidenziando il differente impatto dei pesi pre-addestrati sulla topologia della Loss Function.

I grafici seguenti riassumono l'evoluzione delle metriche per entrambi i modelli dall'epoca iniziale a quella finale:

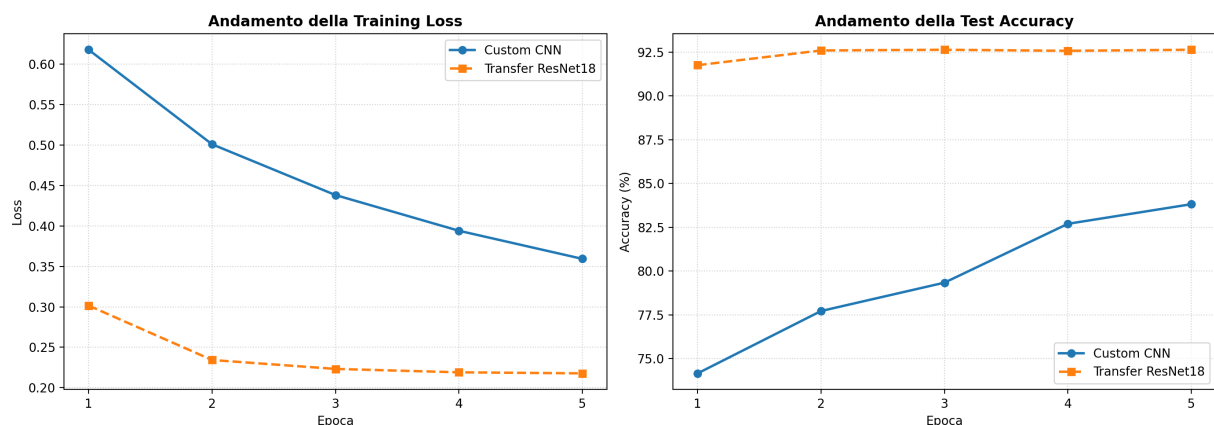


Figura 1: Grafici dell'andamento del Training Loss e della Test accuracy nelle 5 epoche per entrambi i modelli.

Come si evince dai grafici e dai log di addestramento:

- **Custom_CNN:** Mostra un apprendimento costante e progressivo lungo tutte le epoche. La loss scende regolarmente da 0.6176 a 0.3593, mentre l'accuratezza sul test set sale dal 74.16% iniziale fino all'83.82% dell'ultima epoca. Questo andamento lineare descrive una discesa del gradiente regolare e priva di repentine divergenze, indicando che il modello sta estraendo feature locali utili in modo incrementale senza mostrare segni di overfitting precoce, pur risentendo dei limiti strutturali di un'architettura leggera costruita da zero.
- **Transfer_ResNet:** Evidenzia la straordinaria efficacia del Transfer Learning applicato a reti profonde. Già alla prima epoca la rete dimostra una forte convergenza, toccando un'accuratezza del 92.08%. Nelle epoche successive l'apprendimento si stabilizza su un plateau ad alta precisione (92.84% nelle epoche 2 e 3), raggiungendo il picco massimo del 93.04% durante la quarta epoca, per poi registrare una lieve flessione al 92.48% nell'epoca finale.

Questa leggera discesa post-picco non costituisce un principio di overfitting strutturale, bensì descrive il classico fenomeno di *oscillazione (bouncing) attorno al minimo globale* della Loss Function. Poiché il tasso di apprendimento è mantenuto fisso a 0.001, nelle fasi terminali dell’addestramento il passo calcolato dall’algoritmo Adam risulta geometricamente troppo ampio rispetto alla restrittiva curvatura della valle di ottimo. Di conseguenza, l’ottimizzatore “scavalca” il punto di minimo perfetto, risalendo leggermente sul versante opposto della funzione di costo e generando la micro-fluttuazione registrata.

5.2 Analisi Critica delle Matrici di Confusione

Per comprendere la natura geometrica degli errori commessi dalle reti e verificare la presenza di eventuali bias predittivi verso una delle due classi, si analizzano le matrici di confusione generate in fase di valutazione sul test set (composto da un totale di 5.000 immagini).

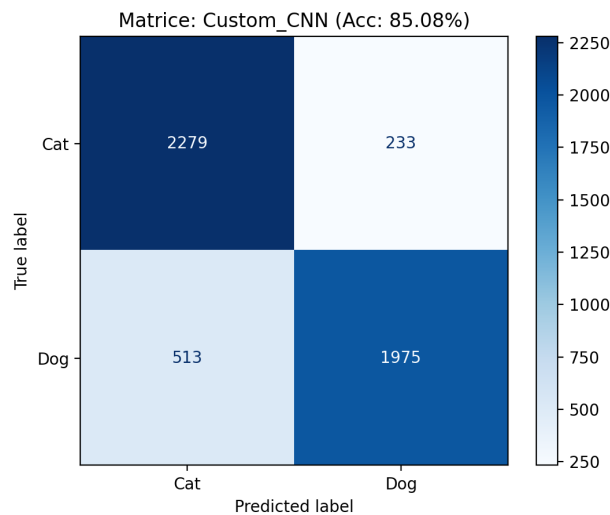


Figura 2: Matrice di Confusione del modello Custom CNN sul Test Set

L’analisi dei singoli quadranti della matrice per la **Custom_CNN** (Figura 2) evidenzia la seguente distribuzione dei campioni:

- **Veri Negativi (Gatti corretti):** 2279 campioni.
- **Veri Positivi (Cani corretti):** 1975 campioni.
- **Falsi Positivi (Gatti predetti come Cani):** 233 campioni.
- **Falsi Negativi (Cani predetti come Gatti):** 513 campioni.

Dall’analisi emerge un parziale sbilanciamento nell’errore: il modello mostra una tendenza più marcata a scambiare i cani per gatti (513 falsi negativi) rispetto al contrario (233 falsi positivi). Questo comportamento suggerisce che le feature morfologiche primitive estratte dai tre layer convoluzionali della rete custom (da pixel entry-level a 128x128) risultano

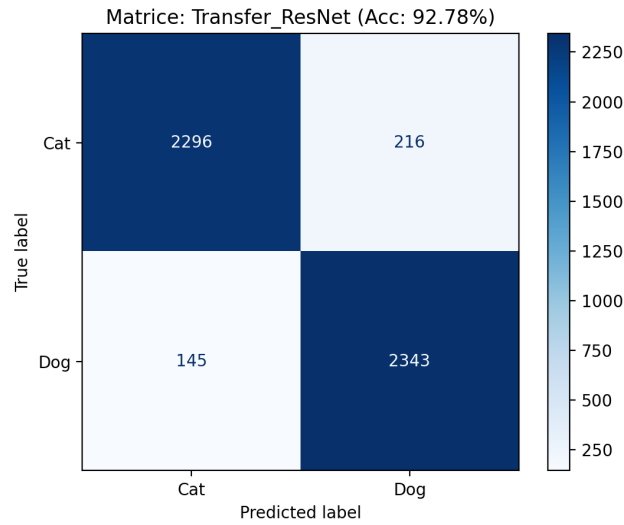


Figura 3: Matrice di Confusione del modello Transfer ResNet18 sul Test Set

parzialmente più conservative o robuste nel riconoscere i pattern di background o di forma tipici della classe "Cat".

Esaminando la matrice della **Transfer_ResNet** (Figura 3), si nota un drastico abbattimento del tasso d'errore complessivo, abbinato a una distribuzione dei pesi decisamente più equilibrata tra le classi:

- **Veri Negativi (Gatti corretti):** 2296 campioni.
- **Veri Positivi (Cani corretti):** 2343 campioni.
- **Falsi Positivi (Gatti predetti come Cani):** 216 campioni.
- **Falsi Negativi (Cani predetti come Gatti):** 145 campioni.

Il modello incorre in errore in appena 361 circostanze su 5.000 immagini totali. In questo scenario, i filtri convoluzionali ereditati da ImageNet dimostrano una capacità di astrazione geometrica e di tessitura (mappatura fine di padiglioni auricolari, regioni oculari e pattern del manto) talmente solida da minimizzare i bias di classificazione. Ciò garantisce eccellenti proprietà di generalizzazione statistica su entrambe le distribuzioni.

6 Conclusioni

L'obiettivo di questo progetto era affrontare il problema della classificazione binaria di immagini tramite reti neurali convoluzionali, valutando le differenze prestazionali, architetture e metodologiche tra un modello progettato ex novo (**Custom_CNN**) e una soluzione basata sul Transfer Learning (**Transfer_ResNet**).

L'analisi sperimentale condotta e i riscontri empirici ottenuti permettono di trarre diverse conclusioni significative:

- **Efficacia del Transfer Learning:** L'ipotesi iniziale secondo cui una rete pre-addestrata avrebbe fornito risultati nettamente superiori si è confermata sul campo. La **Transfer_ResNet** ha dimostrato una superiorità strutturale non solo nell'accuratezza finale stabile al 92.64%, ma soprattutto nella velocità di convergenza. Raggiungere un'accuratezza del 91.76% già al termine della prima epoca dimostra che le feature gerarchiche estratte dal dataset generalista ImageNet sono altamente trasferibili e immediatamente spendibili anche per domini visivi specifici come la differenziazione tra cani e gatti.
- **Validità e Sostenibilità dell'Architettura Custom:** Nonostante il divario rispetto al modello pre-addestrato, lo sviluppo della **Custom_CNN** ha confermato l'adeguatezza delle scelte progettuali. Un'accuratezza finale dell'83.82% per un modello leggero basato su soli tre strati convoluzionali attesta che la gerarchia spaziale e geometrica impostata nei filtri è stata in grado di estrarre in modo autonomo feature primitive robuste, evitando fenomeni precoci di underfitting o overfitting grazie all'azione combinata della Data Augmentation.

In conclusione, l'elaborato evidenzia come nel panorama odierno dell'Intelligenza Artificiale applicata alla Computer Vision la sfida ingegneristica non risieda necessariamente nella computazione da zero di architetture proprietarie, ma nella capacità di selezionare la metodologia corretta in base ai vincoli operativi. Se la rete custom si presenta come un'ottima soluzione accademica e computazionalmente leggera per comprendere l'estrazione dei tratti morfologici di base, il Transfer Learning si conferma la scelta ottimale in scenari reali.